Floating-point Arithmetic and FPU Unit

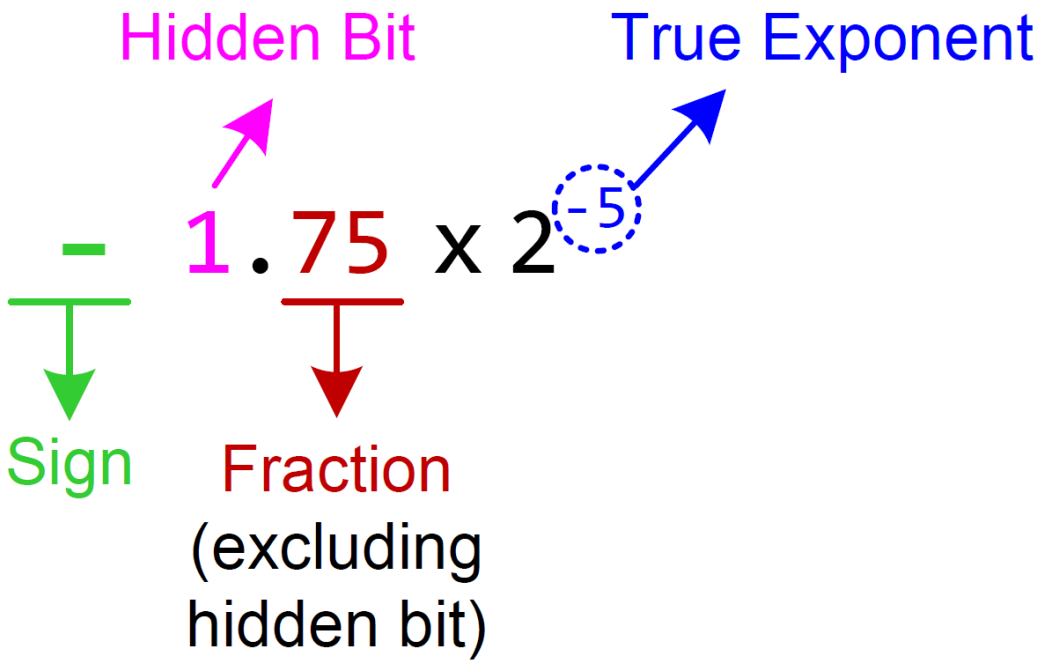
CSE 4095

Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

Taken from Dr. Yifeng Zhu's Lecture Notes

Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 12

Normalization



We can convert 10.746×2^6 to normalized format as follows:

$$10.746\times2^6 = \frac{10.746}{8}\times8\times2^6 = 1.34325\times2^9$$

IEEE Standard 754

Single Precision (32 bits)



Double Precision (64 bits)

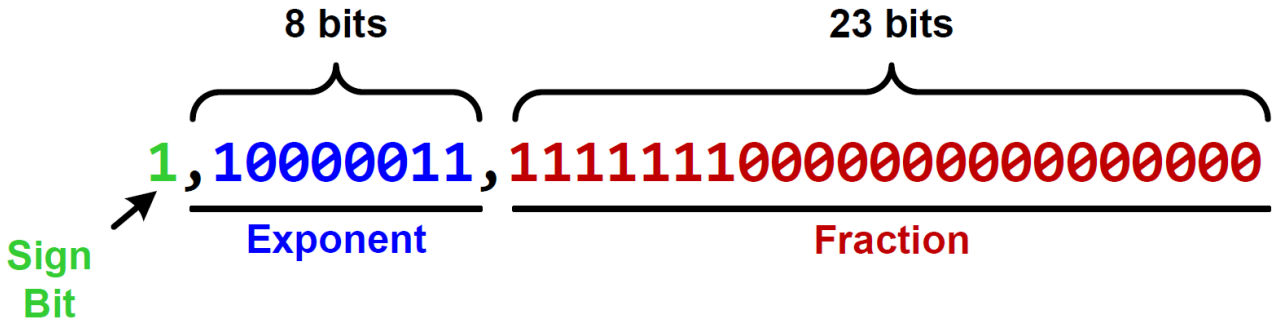


IEEE 754 value:

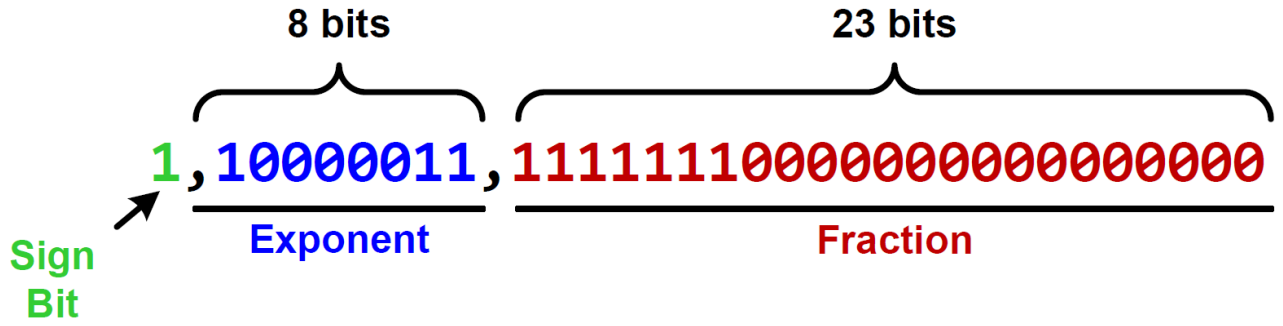
$$(-1)^{S} \times (1 + Fraction) \times 2^{Exponent - Bias}$$

where Bias = **127** for single precision
Bias = **1023** for double precision

Decoding 0xC1FF0000 into a floating-point number

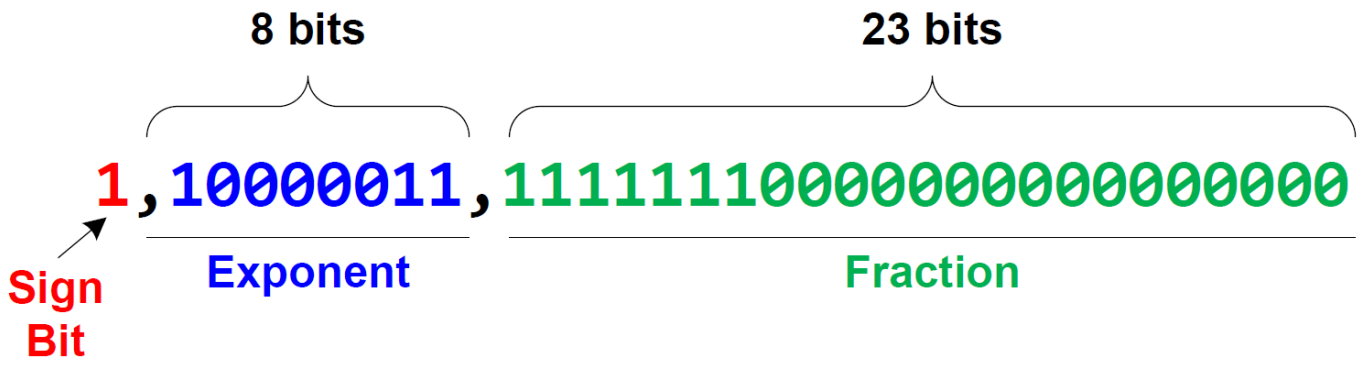


Decoding 0xC1FF0000 into a floating-point number



- Sign = 1
- Exponent = $10000011_2 = 131$
- Fraction = 0.1111111_2

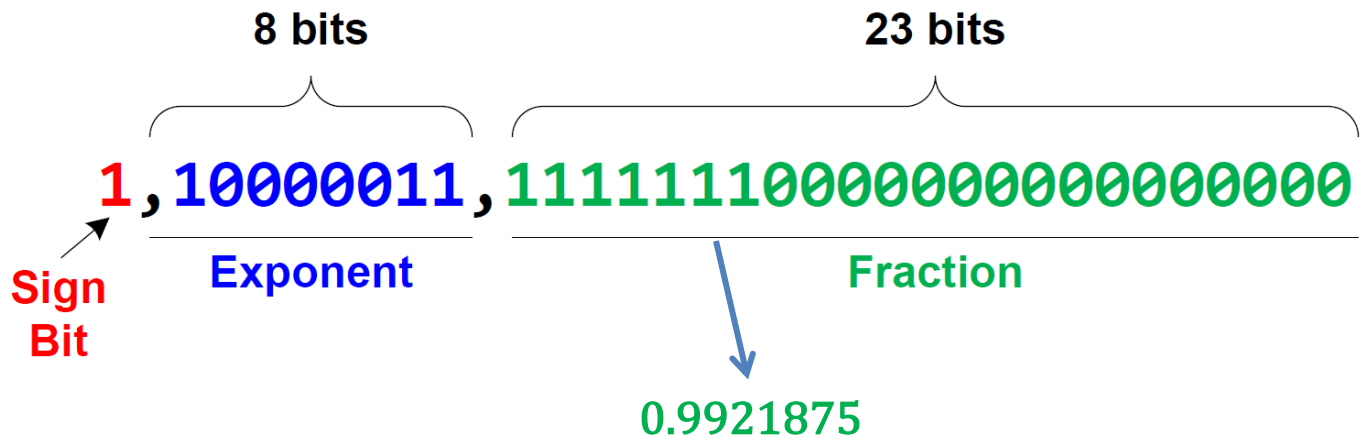
Decoding 0xC1FF0000 into a floating-point number



Fraction

$$0.1111111_2 = 1\times2^{-1} + 1\times2^{-2} + 1\times2^{-3} + 1\times2^{-4} + 1\times2^{-5} + 1\times2^{-6} + 1\times2^{-7}$$
$$= 0.5 + 0.25 + 0.125 + 0.0625 + 0.03125 + 0.015625 + 0.078125$$
$$= 0.9921875$$

Decoding 0xC1FF0000 into a floating-point number



$$
\begin{aligned}
f &= (-1)^S \times (1 + Fraction) \times 2^{Exponent-127} \\
&= (-1)^1 \times (1 + 0.9921875) \times 2^{131-127} \\
&= -1 \times 1.9921875 \times 2^4 \\
&= -31.875
\end{aligned}
$$

Encoding 14.5 into IEEE Std 754 Single-Precision

Normalization:

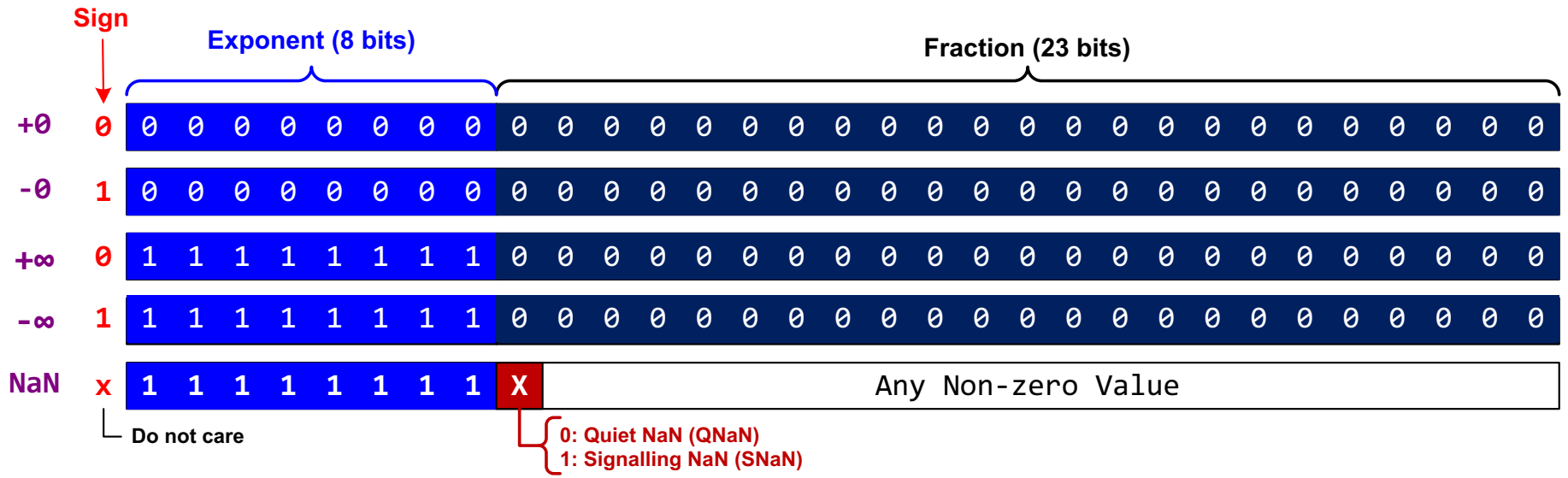
$$14.5 = 1.8125 \times 2^3$$

$$= (1 + 0.8125) \times 2^3$$

- $Sign = 0$
- $Exponent = 3 + 127 = 130 = 1000010_2$
- $Fraction = 0.1101_2$
  - $0.8125 \times 2 = 1.625 = 1 + 0.625$
  - $0.625 \times 2 = 1.25 = 1 + 0.25$
  - $0.25 \times 2 = 0.5 = 0 + 0.5$
  - $0.5 \times 2 = 1$
- 14.5 = 01000001011010000000000000000000 in binary or 0x41680000 in hex

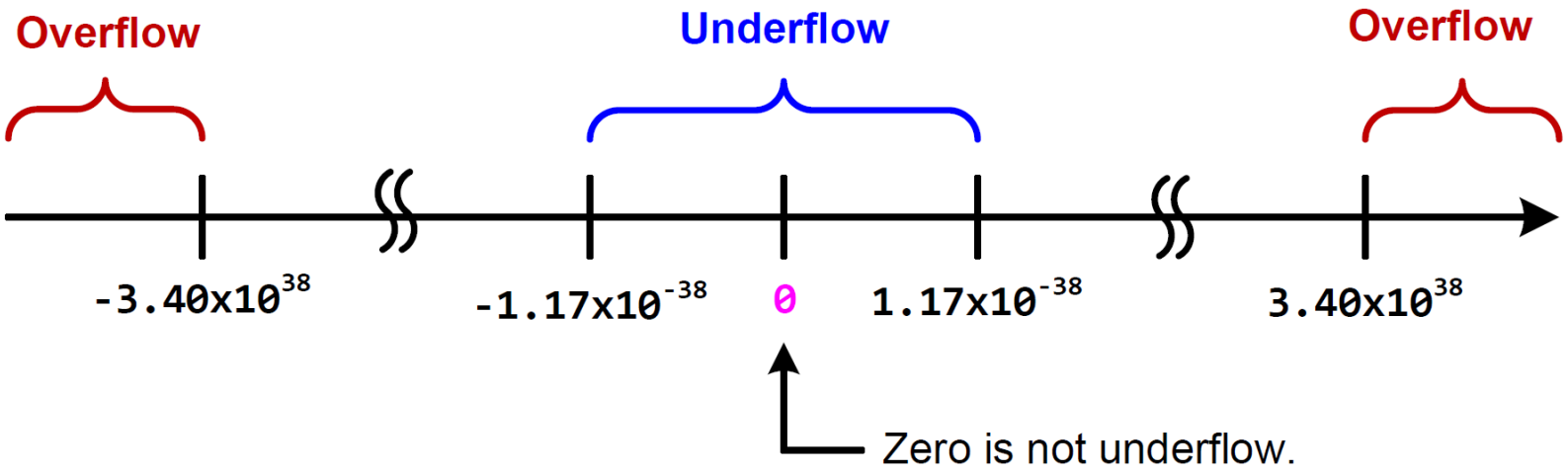
Special Values

- Exponents 00000000 and 111111111 are reserved.



- Example of Not-A-Number (NaN)
  - `log(-10.0)`, `sqrt(-1.0)`, `0.0/0.0`, $-\infty + \infty$,

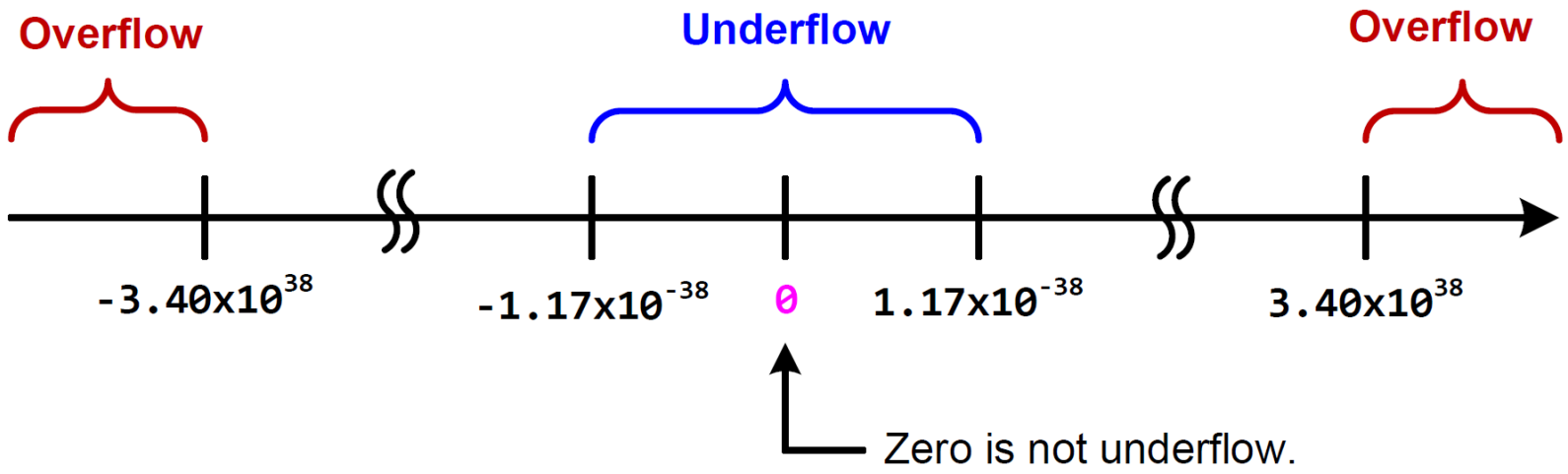
Overflow and Underflow



Numbers closest to zero

$$(-1)^S \times (1+0) \times 2^{1-127} = \pm 2^{-126} \approx \pm 1.18 \times 10^{-38}$$

Overflow and Underflow

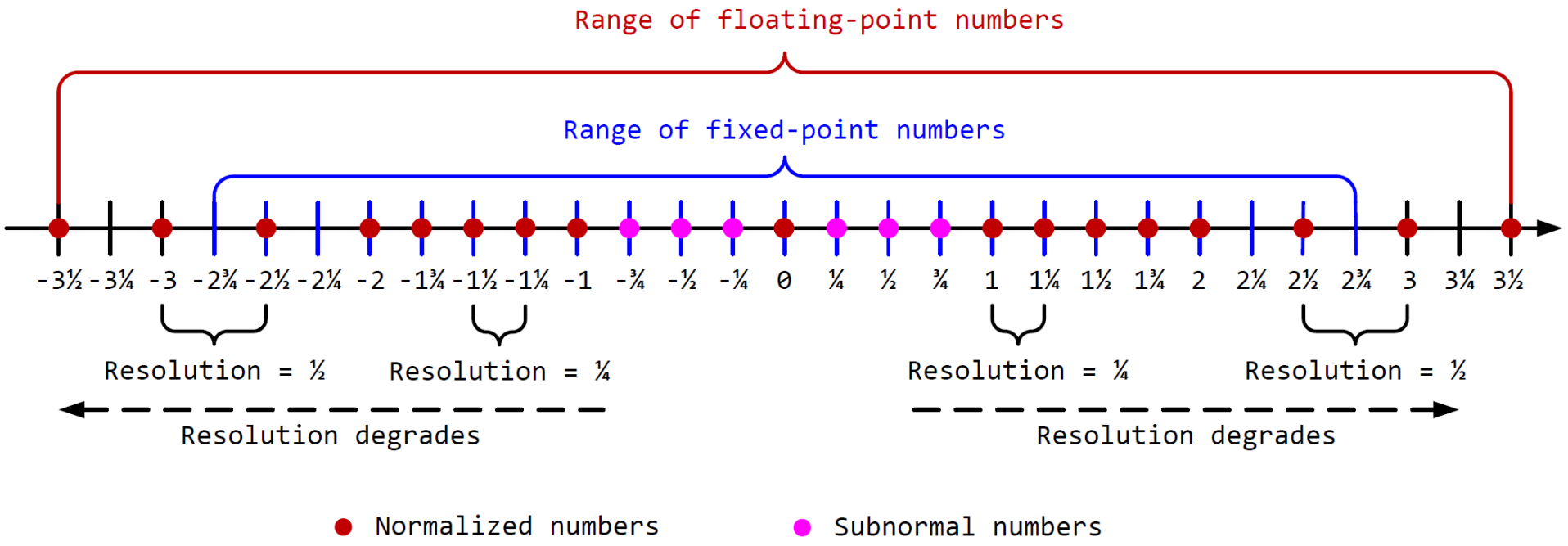


Numbers farthest from zero

$$(-1)^S \times \left(1 + (1 - 2^{-23})\right) \times 2^{254-127} = \pm(2^{128} - 2^{104}) \approx \pm 3.40 \times 10^{38}$$

Resolution

- Given a hypothetical five-bit floating-point system (similar to IEEE 754).
  - the sign bit, an exponent (2 bits), and a fraction (2 bits)



Rounding

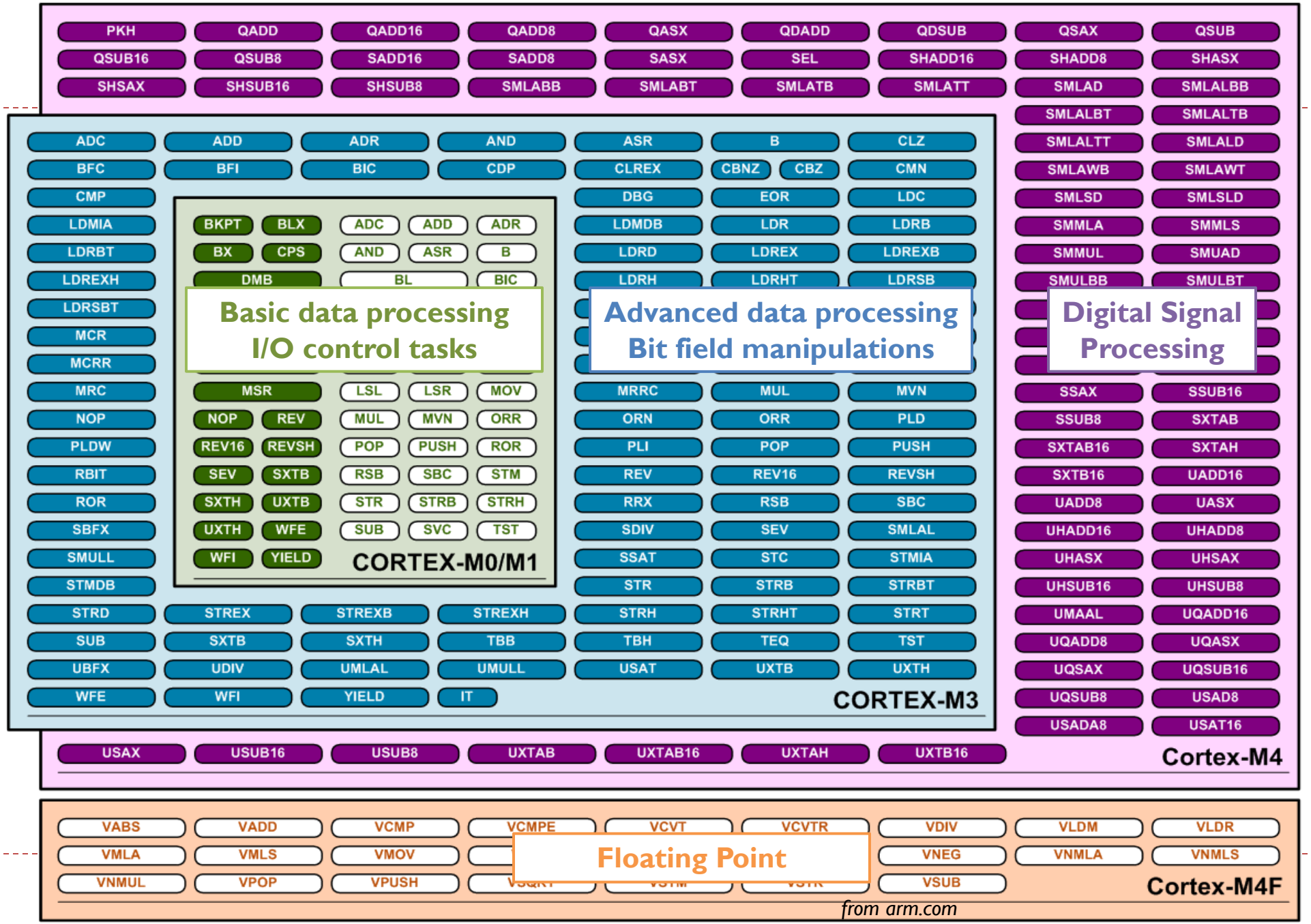
- IEEE 754 defines four different rounding rules:
  - round to the nearest value,
  - round toward zero (truncation),
  - round toward plus infinity (rounding up), and
  - round toward negative infinity (rounding down).

Assume only 5 digits are allowed after the decimal point

Rounding Rule	Data	Rounded Result
Nearest	+0.123456	+0.12346
	-0.123456	-0.12346
Truncate	+0.123456	+0.12345
	-0.123456	-0.12345
Rounding up	+0.123456	+0.12346
	-0.123456	-0.12345
Rounding down	+0.123456	+0.12345
	-0.123456	-0.12346

Floating-point Unit (FPU)

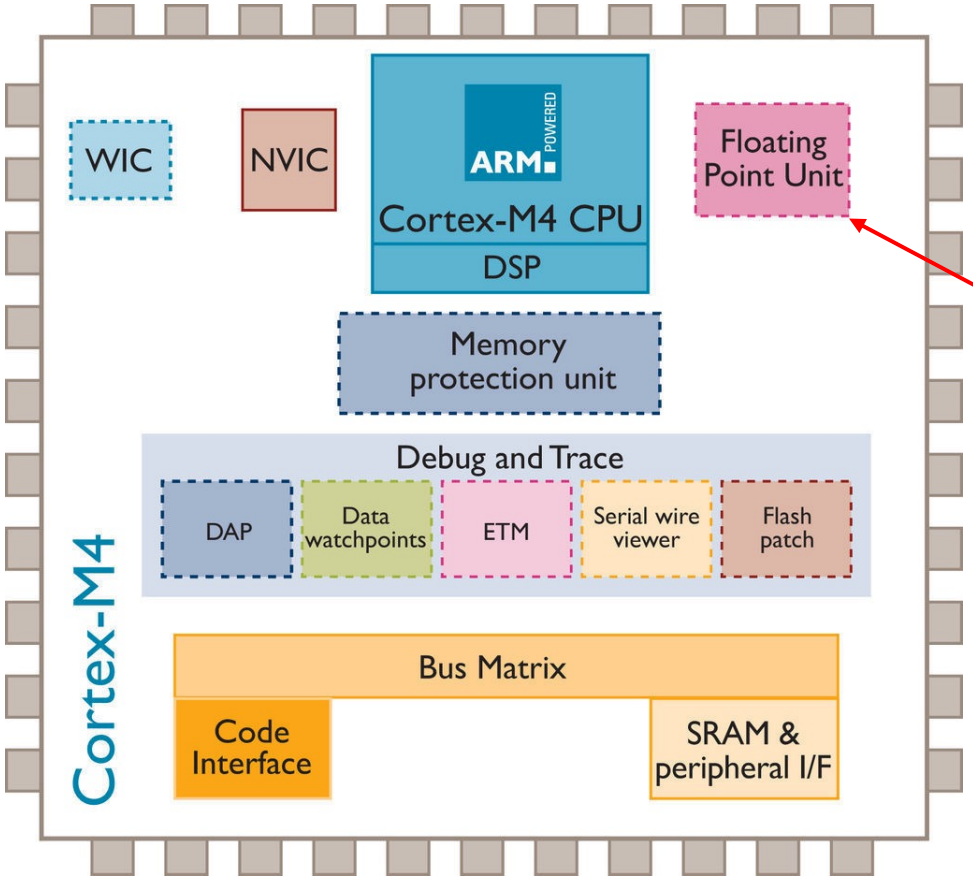
ARM Cortex-M Instruction Sets



from arm.com

FPU on Cortex-M4

Cortex-M4 = Cortex-M3 + DSP + FPU (optional)



FPU is optional

from arm.com

FPU on Cortex-M4

Single Precision (32 bits)

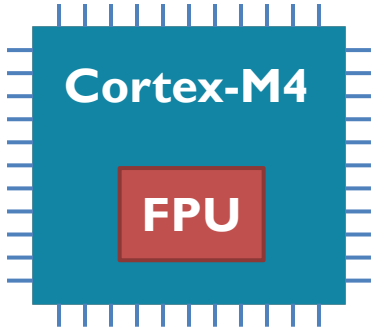
"float" in C



Double Precision (64 bits)

"double" in C



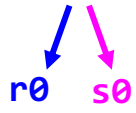


FPU only support single precision.

Floating-point Multiplication

```
float area_of_rectangle(float length, float width) {
    float area;
    area = length * width;
    return area;
}
```





Software-based FP Multiplication

```
; Inputs:
;   r0 = length
;   r1 = width
; Return:
;   r0 = area

area_of_rectangle PROC
  PUSH {lr}
  ; call software library
  BL __aeabi_fmul
  POP {pc}  ; return area in r0

  ENDP
```

Hardware-based FP Multiplication

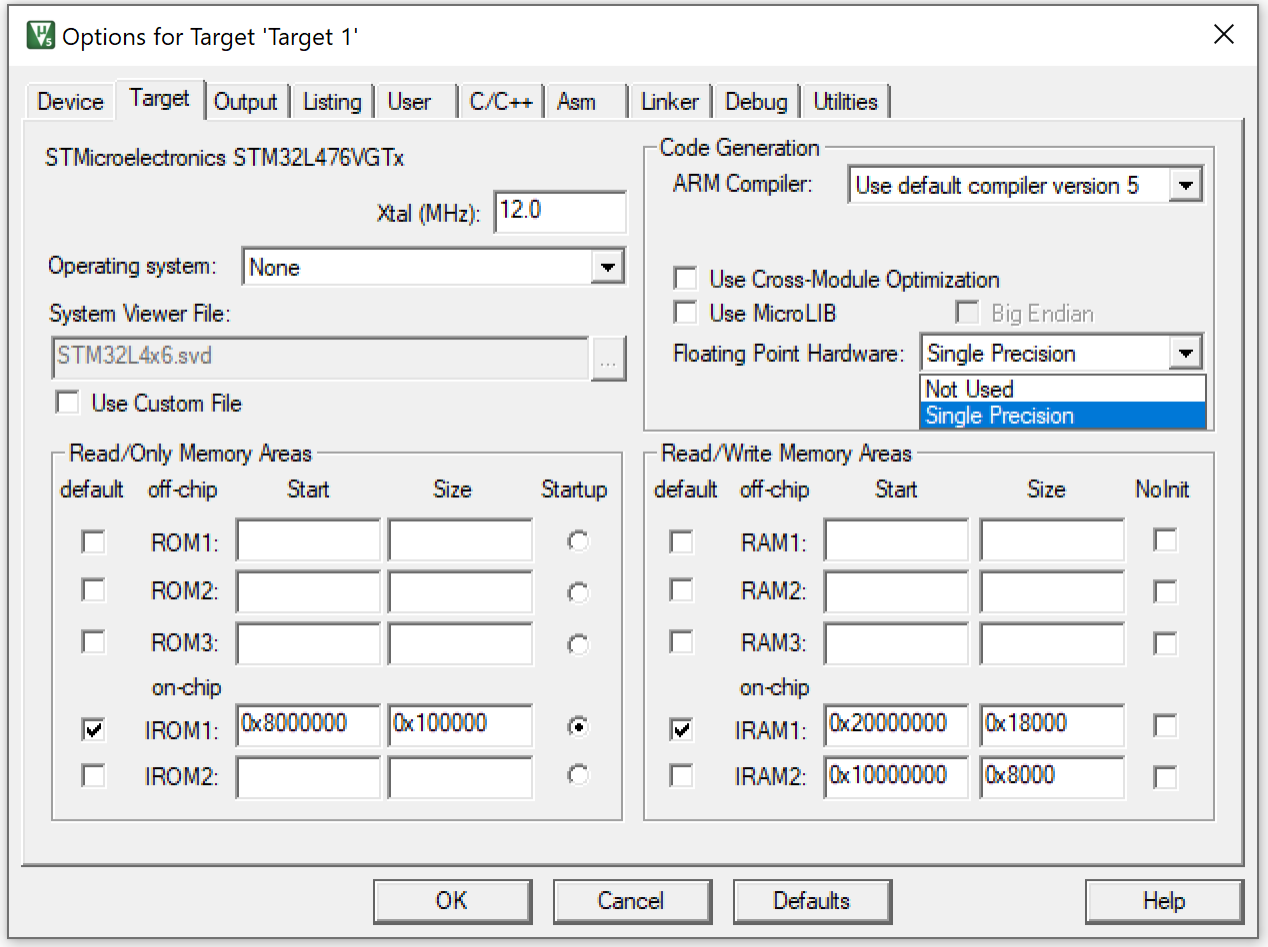
```
; Inputs:
;   s0 = length
;   s1 = width
; Return:
;   s0 = area

area_of_rectangle PROC

  ; Call FPU instructions
  VMUL.F32 s0, s0, s1
  BX lr   ; return area in s0

  ENDP
```

ARM Keil



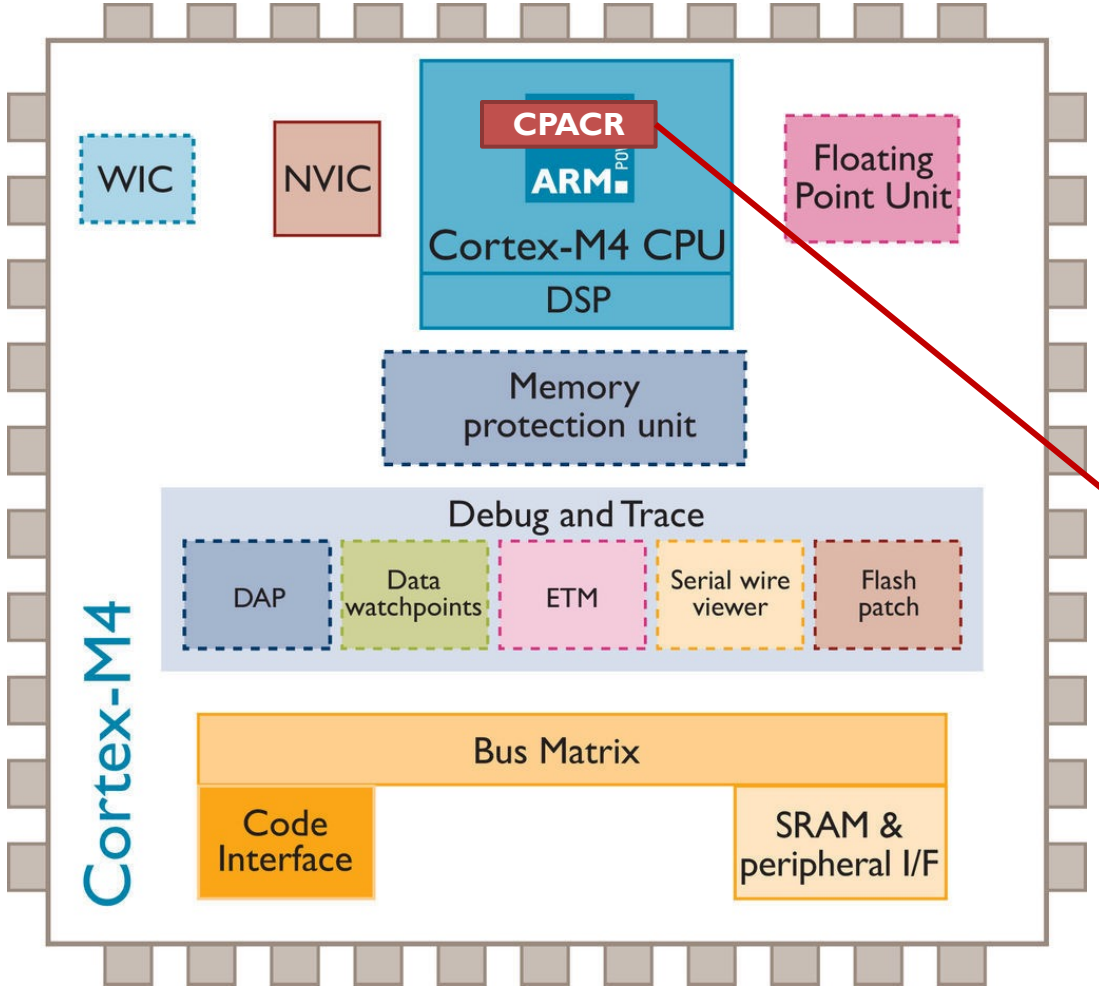
Not Used

- FP operations are performed by software FP library

Single Precision

- Single-precision performed by FPU hardware directly
- Double-precision performed by software FP library

Enable FPU

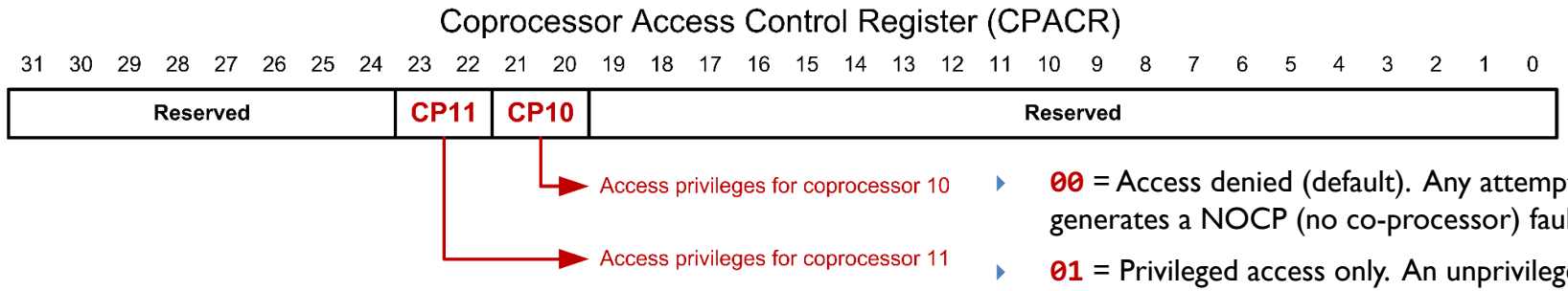


from arm.com

Coprocessor Access Control Register (CPACR) in System Control Block (SCB)

```
SCB->CPACR |= (0xF<<20);
```

Enable FPU

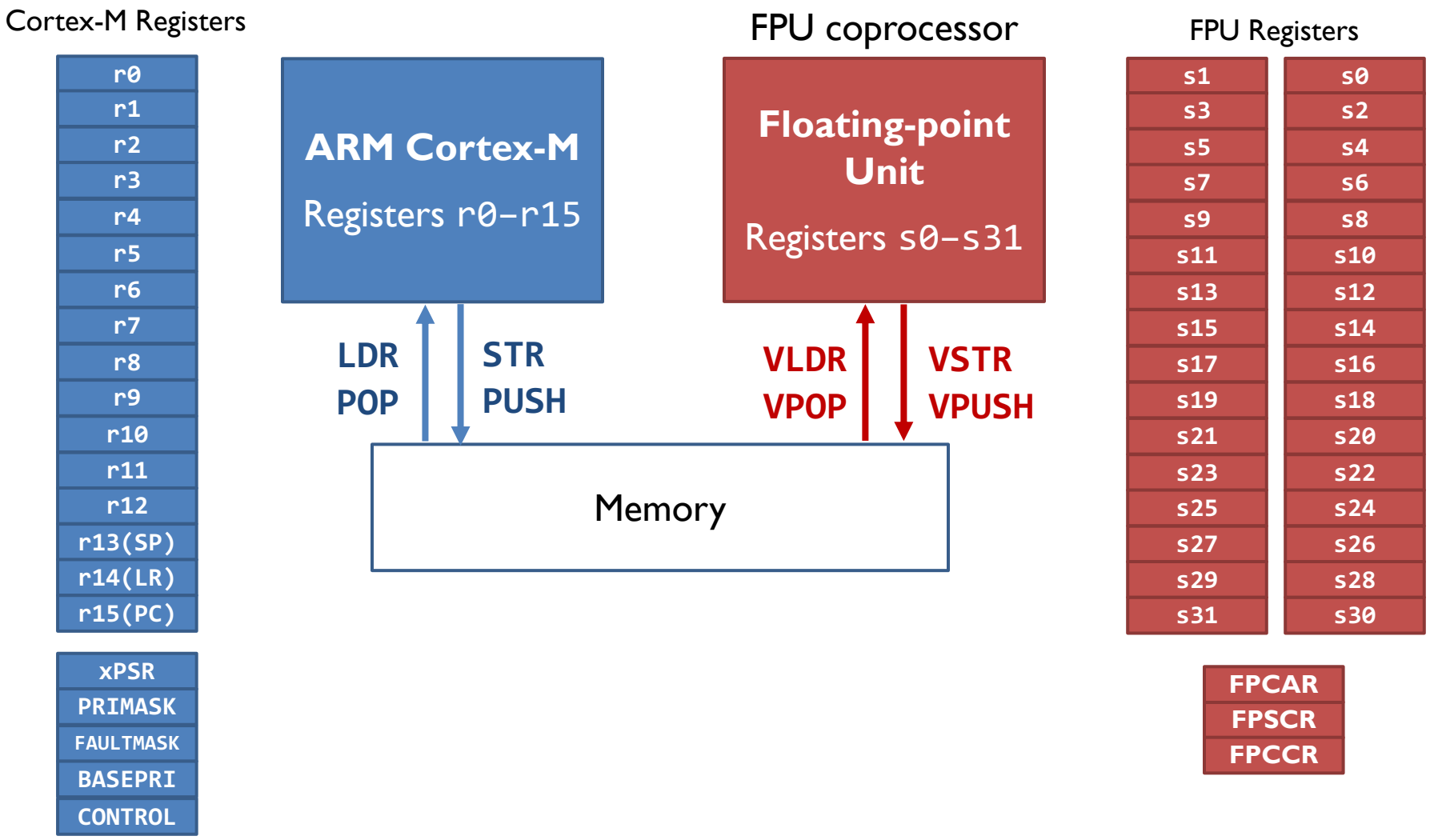


- **00** = Access denied (default). Any attempted access generates a NOCP (no co-processor) fault.
- **01** = Privileged access only. An unprivileged access generates a NOCP fault.
- **10** = Reserved.
- **11** = ***Full access***.

C Program	Assembly Program

`// Enable FPU by setting CP10 and`
`// CP11 fields in System Control`
`// Block (SCB)`
`SCB->CPACR |= (0xF << 20);`
	`; Load SCB base address (0xE000ED00)`
	`LDR r0, = SCB_BASE`
	
	`; Read SCB->CPACR`
	`LDR r1, [r0, #SCB_CPACR]`
	
	`; Enable full access to CP10 and CP11`
	`ORR r1, r1, #(0xF << 20)`
	
	`; Write SCB->CPACR`
	`STR r1, [r0, #SCB_CPACR];`

Floating-point Unit (FPU)



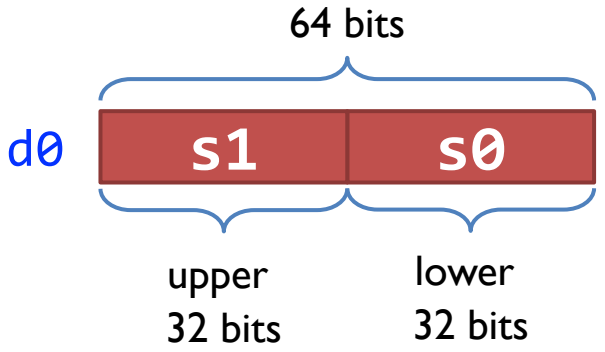


Floating-Point Registers

FPU Registers

d0	s1	s0
d1	s3	s2
d2	s5	s4
d3	s7	s6
d4	s9	s8
d5	s11	s10
d6	s13	s12
d7	s15	s14
d8	s17	s16
d9	s19	s18
d10	s21	s20
d11	s23	s22
d12	s25	s24
d13	s27	s26
d14	s29	s28
d15	s31	s30

s = single precision (32 bits)
d = double precision (64 bits)



Load a float from address in r0

```
VLDR.F32 s0, [r0]
```

Load a double from address in r0

```
; stored into s1 and s0
VLDR.F64 d0, [r0]
```

Stack operations

```
VPUSH.64 {d0,d1}
VPUSH.32 {s4,s6}
```

FPU Instructions

	ARM Cortex-M	FPU
Accessing memory	LDR, STR, LDM, STM, PUSH, POP	VLDR, VSTR, VLDM, VSTM, VPUSH, VPOP
Copying registers	MOV, MRS, MSR	VMOV, VMRS, VMSR
Arithmetic	ADD, SUB, MULS, SDIV/UDIV,	VADD, VSUB, VMUL, VDIV
Multiply-Accumulation	MLA, MLS	VMLA, VMLS
Comparison	CMP	VCMP
Advanced Arithmetic		VABS, VNEG, VSQRT

```
VMLA Sd, Sn, Sm ; Sd = Sd + Sn * Sm
VMLS Sd, Sn, Sm ; Sd = Sd - Sn * Sm
```

Load and Store Floating-point Numbers

`VLDR.F32 Sd, [Rn]`	Load one single-precision float
`VLDR.F64 Dd, [Rn]`	Load one double-precision float
`VSTR.32 Sd, [Rn]`	Store one float registers
`VSTR.64 Dd, [Rn]`	Store one double register
`VLDM.32 Rn{!}, list`	Load multiple single-precision floats
`VLDM.64 Rn{!}, list`	Load multiple double-precision floats
`VSTM.32 Rn{!}, list`	Store multiple float registers
`VSTM.64 Rn{!}, list`	Store multiple double registers
`VPOP.32 list`	Pop float registers from stack
`VPOP.64 list`	Pop double registers from stack
`VPUSH.32 list`	Push float registers to stack
`VPUSH.64 list`	Push double registers to stack

Examples:

```
VLDR.F32 s0, [r0, #4]  ; Pre-index, s0 = mem[r0 + 4]
VLDR.F32 s0, [r0], #4  ; Post-index, s0 = mem[r0], r0 += 4
VLDR.F32 s0, [r0, #4]! ; Pre-index with update, s0 = mem[r0 + 4], r0 += 4
VLDM.64 r0, {d0-d7}    ; Load 8 double precision numbers
VSDM.64 r0, {d0-d7}    ; Store 8 double precision numbers
VPUSH.32 {s16-s31}     ; Save floating point registers
VPOP.32  {s16-s31}     ; Restore floating point registers
```

Copy Floating-point Numbers

`VMOV.F32 Sd, #imm`	Move immediate to float-register
`VMOV.F32 Sd, Sm`	Copy from single-precision register to single-precision register
`VMOV.F64 Dd, Dm`	Copy from double-precision register to double-precision register
`VMOV Sn, Rt`	Copy ARM core register to float register
`VMOV Sm1, Sm2, Rt1, Rt2`	Copy 2 ARM core registers to 2 float registers
`VMOV Dd[x], Rt`	Copy ARM core register to a half of a double-precision floating-point register, where x is 0 or 1.
`VMOV Rt, Dn[x]`	Copy a half of a double-precision floating-point register to ARM core register, where x is 0 or 1.

Examples:

```
VMOV s0, s1, r0, r1    ; s0 = r0, s1 = r1
VMOV r1, d0 [1]        ; r1 = d0 [1] = s1
VMOV.F32 s1, #3.25     ; s1 = 3.25
```

Single-precision Arithmetic Operations

VADD.F32	{Sd,} Sn, Sm	Add floating points, Sd = Sn + Sm
VSUB.F32	{Sd,} Sn, Sm	Subtract float, Sd = Sn - Sm
VDIV.F32	{Sd,} Sn, Sm	Divide single-precision floats, Sd = Sn/Sm
VMUL.F32	{Sd,} Sn, Sm	Multiply float, Sd = Sn * Sm
VNMUL.F32	{Sd,} Sn, Sm	Negate and multiply float, Sd = -1 * Sn * Sm
VNEG.F32	Sd, Sm	Negate float, Sd = -1 * Sm
VABS.F32	Sd, Sm	Absolute value of floats, Sd = |Sm|
VSQRT.F32	Sd, Sm	Square-root of float

C Function

```
float area_of_circle(float radius){
  float PI = 3.14;
  float area;
  area = PI * radius * radius;
  return area;
}
```

Assembly Function

```
area_of_circle PROC
 EXPORT area_of_circle
 VLDR.F32 s1,=3.14 ; Pseudo-instruction
 VMUL.F32 s1,s1,s0 ; s0 = radius
 VMUL.F32 s0,s1,s0 ; return area in s0
 ENDP
```

Multiply-Accumulation Instructions

VLMA.F32 {Sd,} Sn, Sm	Multiply float, then accumulate float, Sd = Sd + Sn*Sm
VLMS.F32 {Sd,} Sn, Sm	Multiply float, then subtract float, Sd = Sd - Sn*Sm
VNMLA.F32 Sd, Sn, Sm	Multiply float, then accumulate, then negate float, Sd = -1 * (Sd + Sn * Sm)
VNMLS.F32 Sd, Sn, Sm	Multiply float, then subtract, then negate float, Sd = -1 * (Sd - Sn * Sm)

Fused MAC instructions: intermediate result are not rounded. ⟹ Better accuracy

VFMA.F32 {Sd,} Sn, Sm	Multiply (fused) then accumulate float, Sd = Sd + Sn*Sm
VFMS.F32 {Sd,} Sn, Sm	Multiply (fused) then subtract float, Sd = Sd - Sn*Sm
VFNMA.F32 {Sd,} Sn, Sm	Multiply (fused) then accumulate then negate float, Sd = -1 * Sd + Sn * Sm
VFNMS.F32 {Sd,} Sn, Sm	Multiply (fused) then subtract then negate float, Sd = -1 * Sd - Sn * Sm

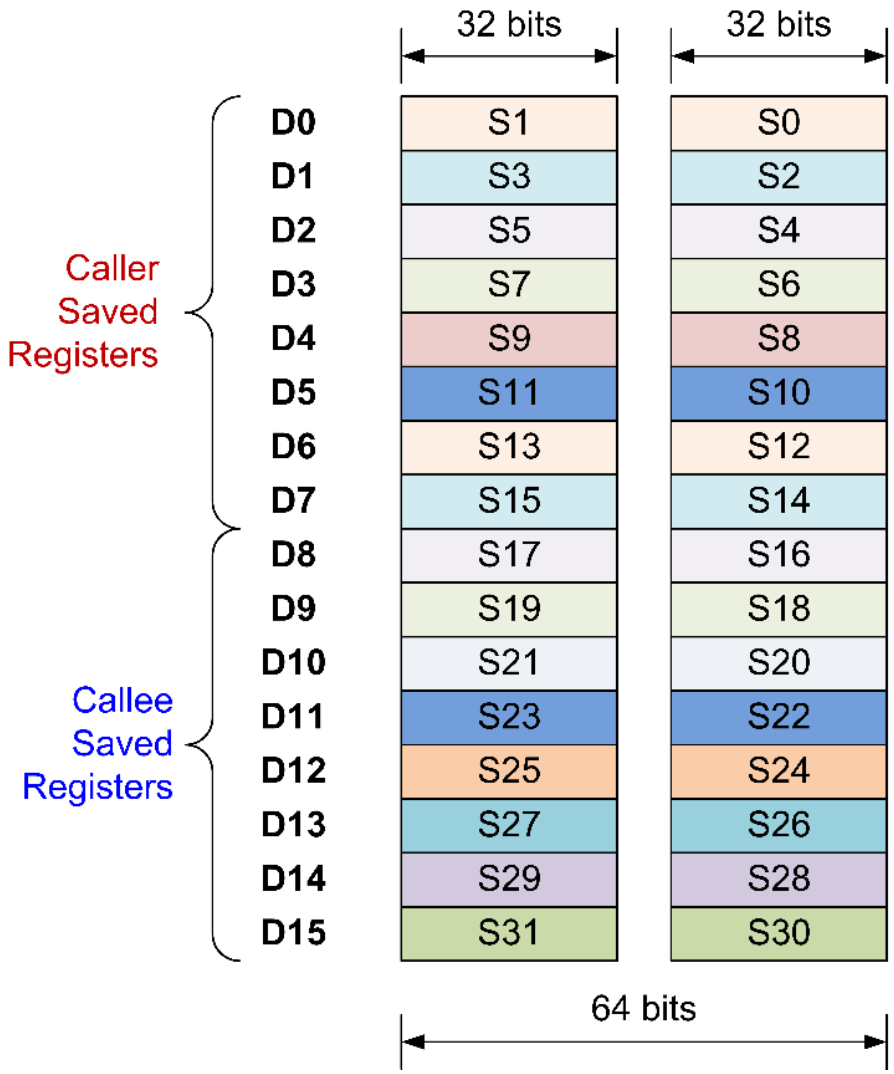
Single-precision comparisons

`VCMP.F32 Sd, <Sm | #0.0>`	Compare two floating-point registers, or one floating-point register and zero
`VCMPE.F32 Sd, <Sm | #0.0>`	Compare two floating-point registers, or one floating-point register and zero, and raise exception for a signaling NaN

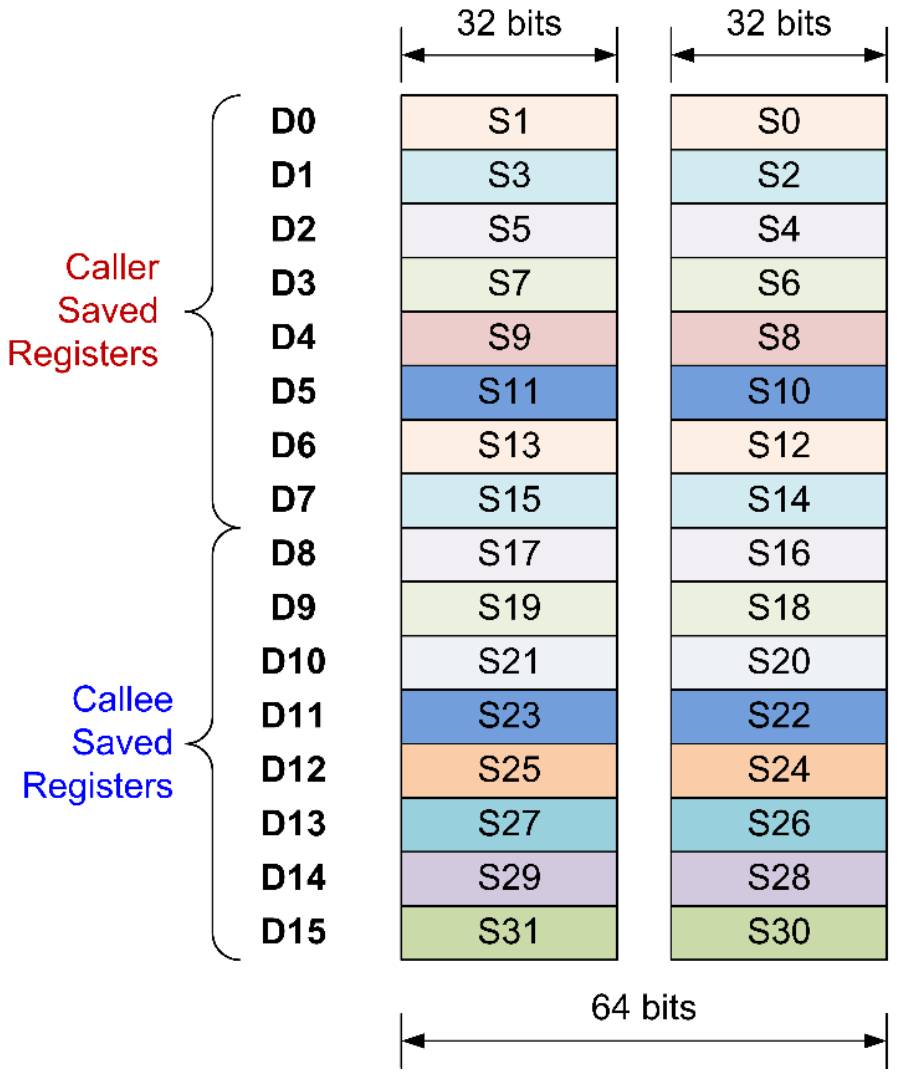
C Program	Assembly Program

`float max(float a, float b){`
`  if (a > b)`
`  return a;`
`  else`
`  return b;`
`}`
	`max PROC`
	`  VCMPE.F32 s0, s1`
	
	`  ; Copy NZCV flags to APSR`
	`  VMRS  APSR_nzcv, FPSCR`
	
	`  BGT  exit`
	`  VMOV.F32  s0, s1`
	
	`exit  BX LR`
	`  ENDP`

Register Usages across Function Calls



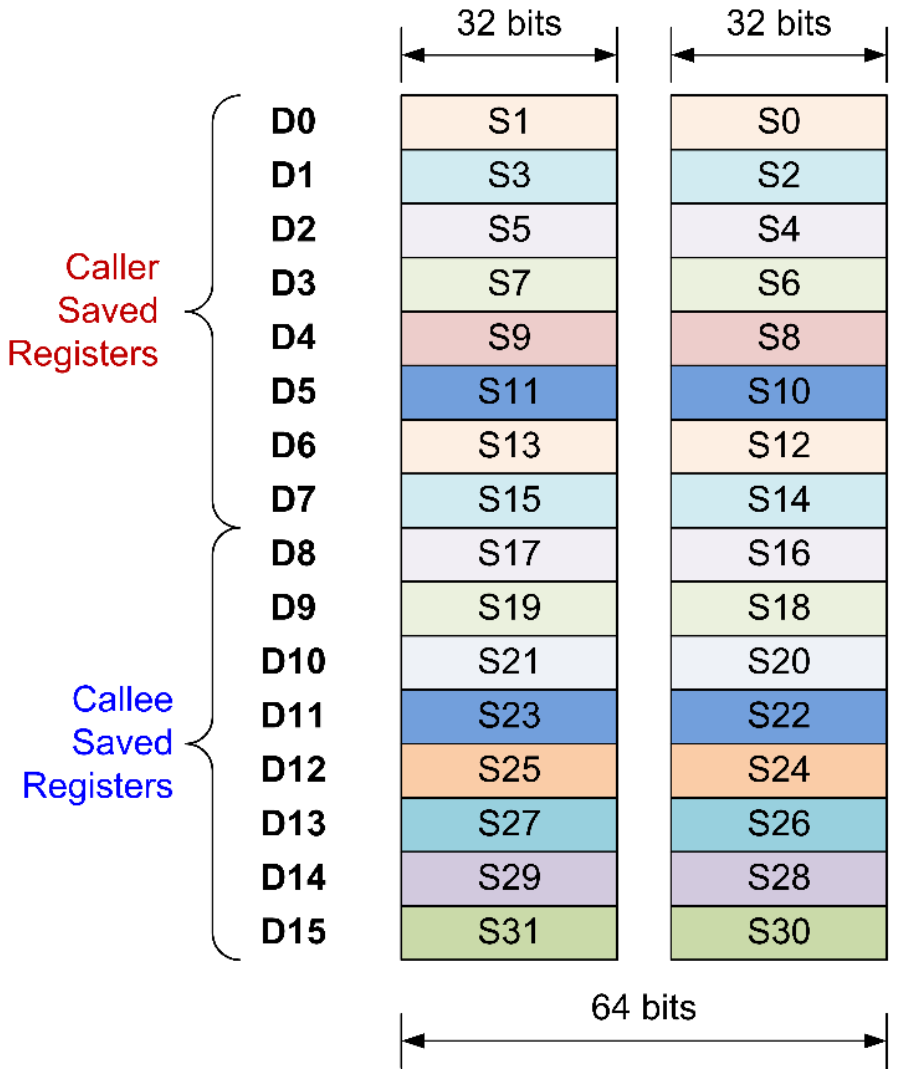
Register Usages across Function Calls




```
A PROC
   ...
   VPUSH {s3, s5}
   BL B ; call function B
   VPOP {s3, s5}
   ...
   ENDP
```

```
B PROC
   ...
   ENDP
```

Register Usages across Function Calls

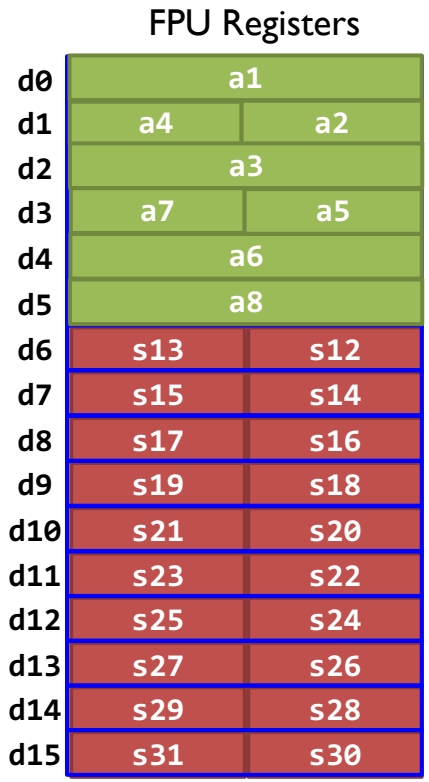



```
A PROC
  ...
  VPUSH {s3, s5}
  BL B ; call function B
  VPOP {s3, s5}
  ...
  ENDP
```

```
B PROC
  VPUSH {s17}
  VLDR.F32 s17, [r0]
  ...
  VPOP {s17}
  ENDP
```

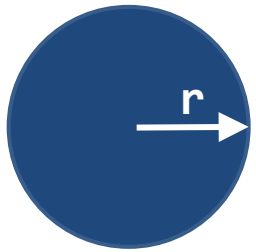
Passing Arguments to a Procedure

- Each argument is assigned in turn to the next free register of the corresponding type
- Example:
  - `double fun(double a1, float a2, double a3, float a4, float a5, double a6, float a7, double a8)`



Example 1: Area of Circle

$$area = \pi r^2$$




```
float area(float radius){
   float pi = 3.14;
   float area;
   area = pi * radius * radius;
   return area;
}
```

Implementation in assembly

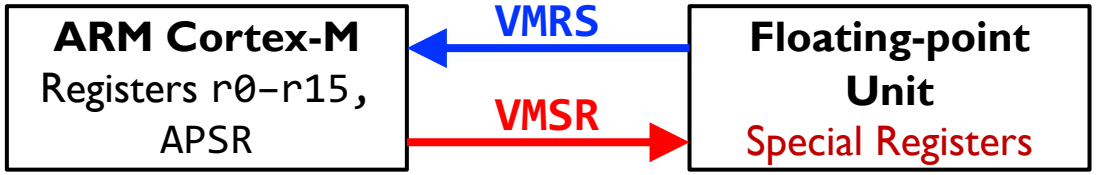
```
area PROC
   EXPORT area
   VLDR.F32 s1, =3.1416 ; Pseudo-instruction
   VMUL.F32 s1, s1, s0  ; r0 = radius (input argument)
   VMUL.F32 s0, s1, s0  ; return area in s0
   BX LR
   ENDP
```

Transfer data between FPU and ARM



Example:

```
VMOV r0, s0  ; Copy s0 into r0
VMOV s1, r1  ; Copy r1 into s0
```



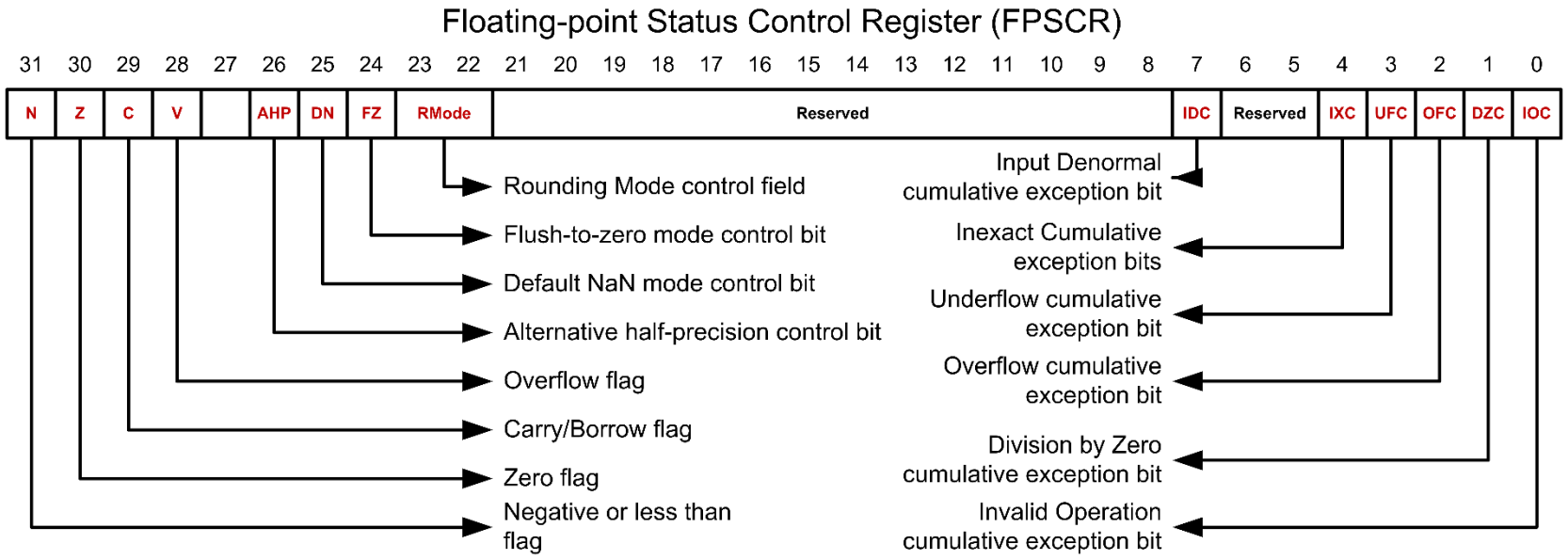
FPSCR, FPCAR, FPCCR

VMRS: Move into ARM register from FPU special register
VMSR: Move into FPU special register from ARM register

Example: Change the rounding mode to round toward zero

```
VMRS r0, FPSCR          ; Read current FPSCR
ORR  r0, r0, #(3<<22)   ; Set bits 22 and 23 to 1
VMSR FPSCR, r0          ; Save into FPSCR
```

Floating-point Status Control Register (FPSCR)



`FPSCR[23:22]`

- **`00`**: Round to nearest (default)
- **`01`**: Round towards $+\infty$
- **`10`**: Round towards $-\infty$
- **`11`**: Round towards `0`

Example 2: Comparison

Copy N, Z, C,V flags from FPU status register to ARM core APSR




```
VMRS APSR_nzcv, FPSCR
```

```
float max(float a, float b){
  if (a > b)
    return a;
  else
    return b;
}
```

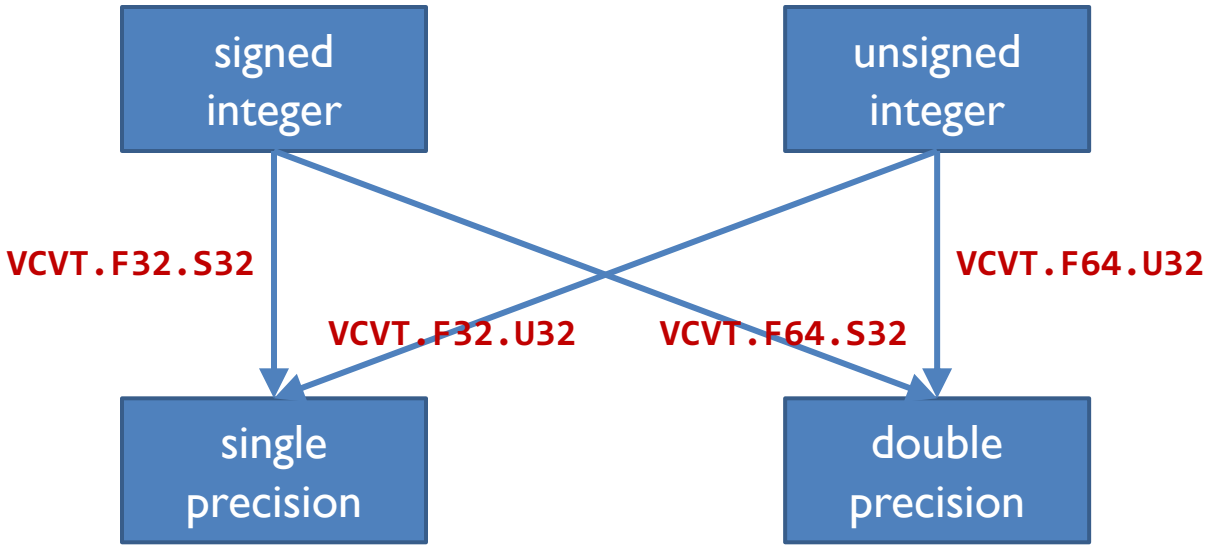
```
max   PROC
      ; Update NZCV in FPSCR
      VCMP.F32 s0, s1

      ; Copy NZCV flags to APSR
      VMRS APSR_nzcv, FPSCR

      VMOVLE.F32  s0, s1

      ENDP
```

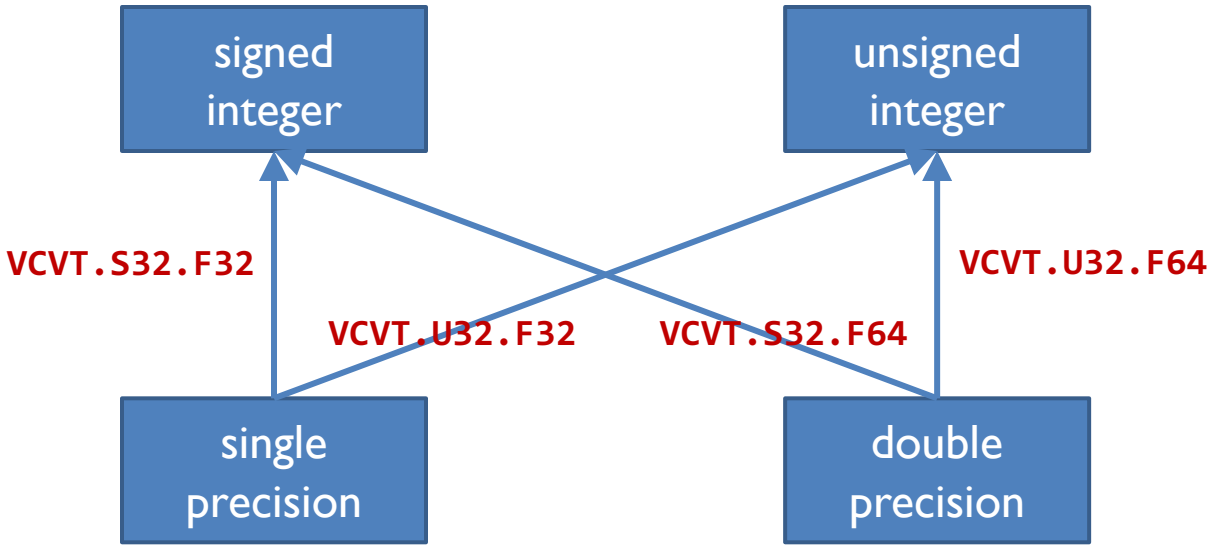
Convert Integer to Floating Point



Example:

```
VCVT.F64.U32  d2, s0  ; Convert unsigned integer to double
```

Convert Floating Point to Integer



Example:

```
VCVT.U32.F64  s0, d2 ; Convert double to unsigned integer
```

Convert between Fixed-Point and Single Precision

```
VCVT.Td.F32 Sd, Sm, #n
VCVT.Td.F32 Sd, Sd, #n
VCVT.F32.Td Sd, Sm, #n
VCVT.F32.Td Sd, Sd, #n
```

- N: the number of fraction bits in fixed point
- Td: type and size of fixed point
  - S32: signed 32-bit value
  - U32: unsigned 32-bit value
  - S16: signed 16-bit value
  - U16: unsigned 16-bit value

Example:

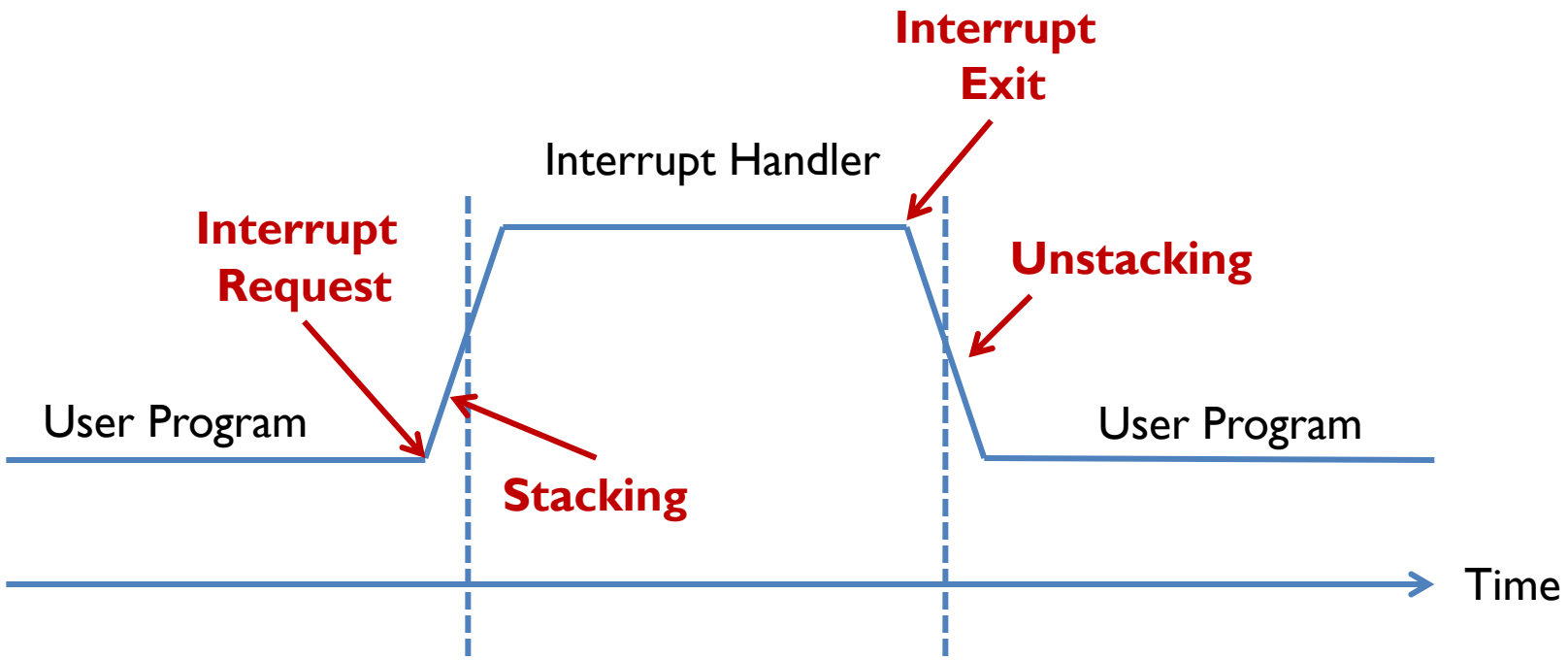
```
VCVT.F32.S32  s0, s0, #10 ; Convert a 32-bit fixed point to float
                          ; Fraction part of the fixed point has 10 bits
```

Precision Conversion

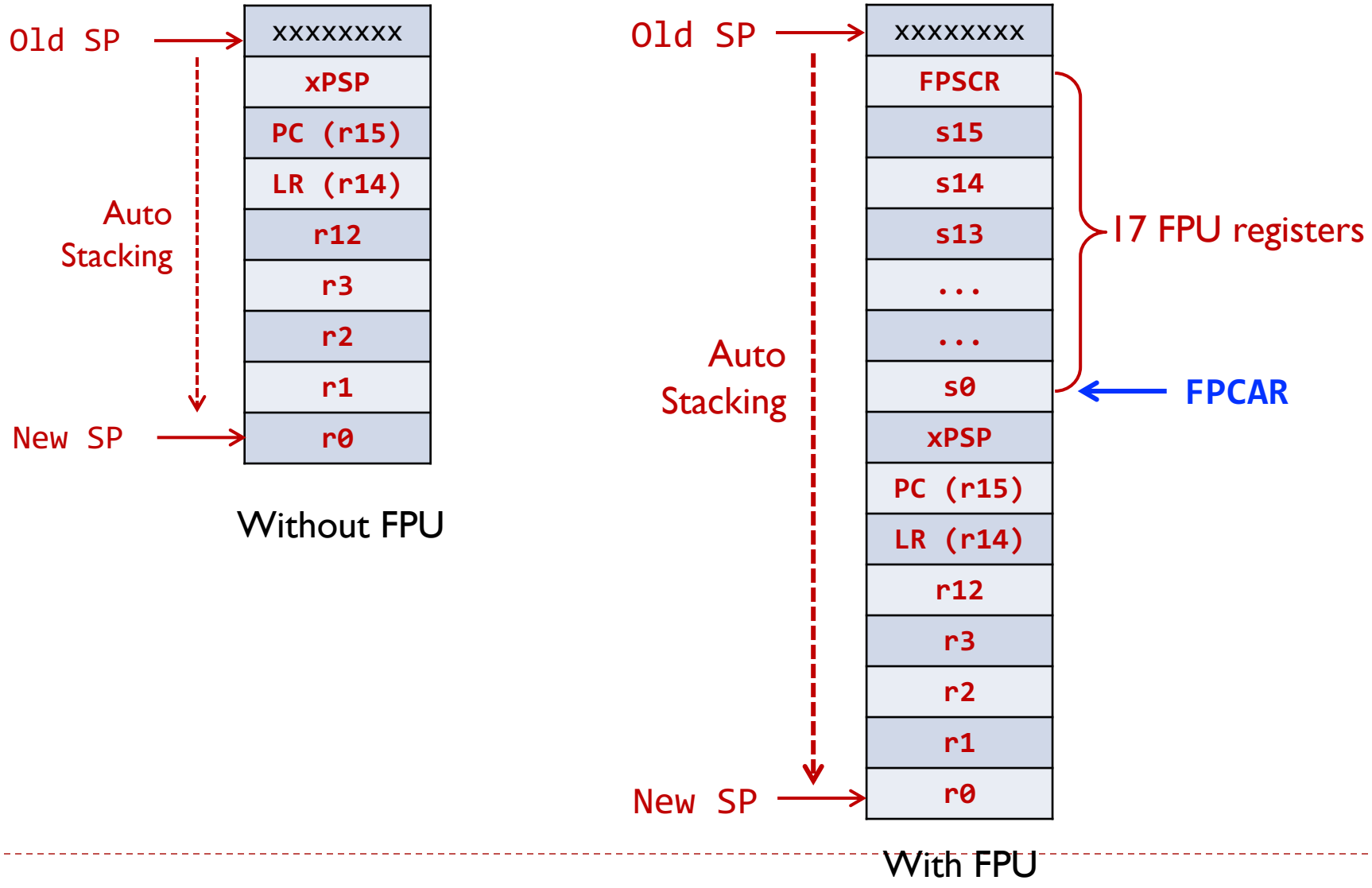
VCVT{R}.S32.F32 Sd, Sm	Convert from single-precision float to signed 32-bit (S32) or unsigned 32-bit (U32) integer.
VCVT{R}.U32.F32 Sd, Sm	
VCVT{R}.F32.S32 Sd, Sm	Convert to single-precision float from signed 32-bit (S32) or unsigned 32-bit (U32) integer.
VCVT{R}.F32.U32 Sd, Sm	
VCVT{R}.Td.F32 Sd, Sm, #n	Convert between single-precision and fixed-point. Td can be S16 (signed 16-bit), U16 (unsigned 16-bit), S32 (signed 32-bit), and U32 (unsigned 32-bit). #n is the number of fraction bits in the fixed-point number.
VCVT{R}.Td.F32 Sd, Sd, #n	
VCVT{R}.F32.Td Sd, Sm, #n	
VCVT{R}.F32.Td Sd, Sd, #n	
VCVT<B|T>.F32.F16 Sd, Sm	Converts half-precision float to single-precision (B = bottom half of Sm, T = top half of Sm)
VCVT<B|T>.F16.F32 Sd, Sm	Converts single-precision float to half-precision (B = bottom half of Sd, T = top half of Sd)

{R}: R is optional, and it means conversion with Rounding

Automatic Stacking & Unstacking



Automatic Stacking & Unstacking

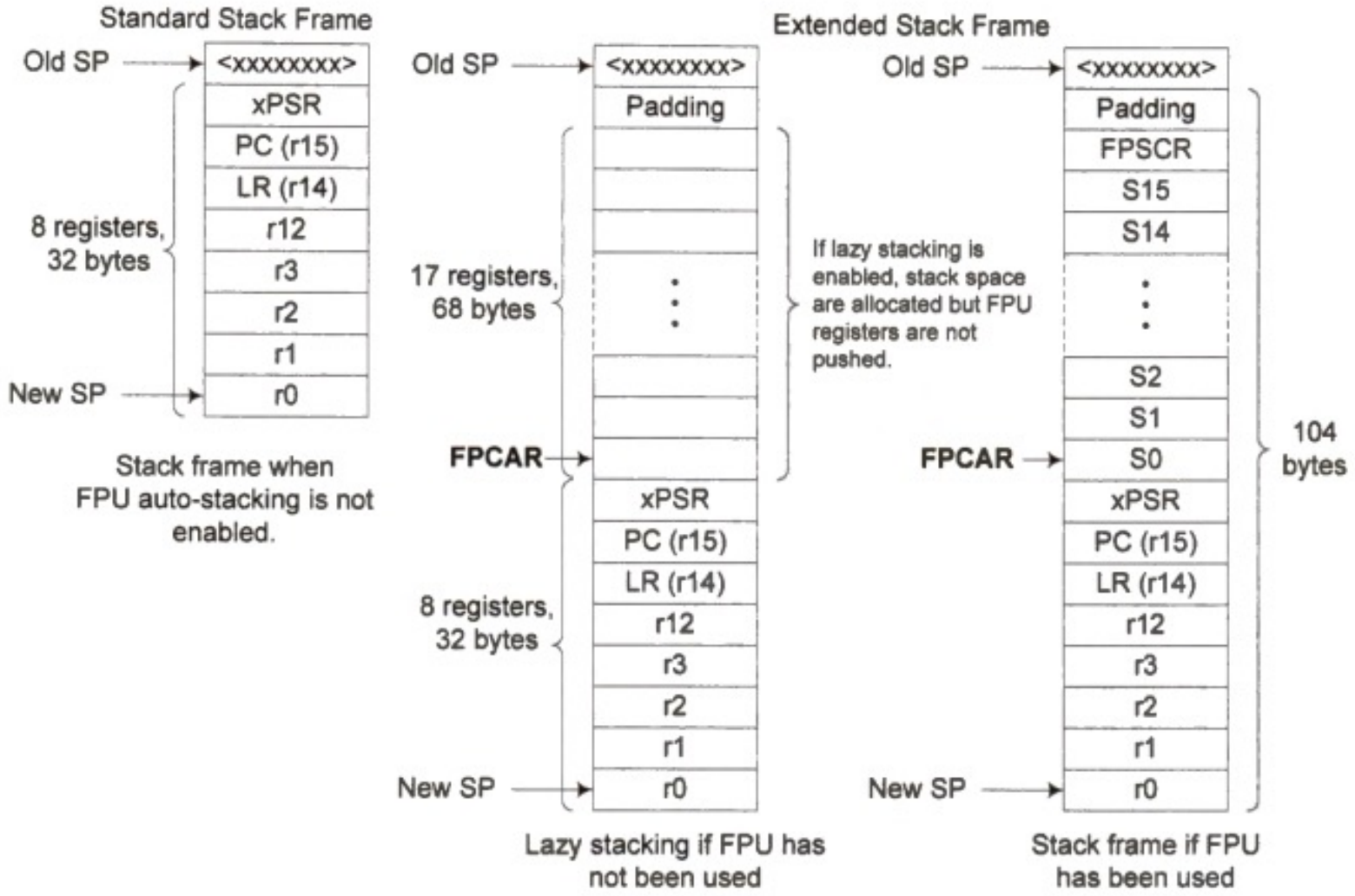


Floating-point Context Address Register (FPCAR)

- The processor stores the stack location holding the value of register s0 to the floating-point context address register (FPCAR).

- Additionally, the processor uses ***lazy stacking*** to reduce the interrupt latency.
- The lazy stacking allows the processor to skip the stacking of FPU registers, if not required, i.e.,
  - if the interrupt handler does not use FPU, or
  - if the interrupted program has not used FPU.

Lazy Stacking



Example: Look up a Float Array

C Code

```
int32_t lookup(float *fArray,
  float f, uint32_t size){

  int i;

  for (i = 0; i < size; i++) {

    if (fArray[i] == f)
      return i;


  }

  return -1;

}
```

Assembly Code

```
lookup PROC
      EXPORT lookup ; Make Lookup visible

      MOV r2, r0 ; r0 = array address
      MOV r3, #0 ; Loop index i = 0
      B check

loop ; Find address of fArray[i]
      ADD r0, r2, r3, LSL #2
      VLDR s1, [r0, #0] ; s1 = fArray[i]

      ; Compare f and fArray[i]
      VCMP.F32 s1, s0
      VMRS APSR_nzcv, FPSCR ; copy flags

      ; Return index i if equal
      MOVEQ r0, r3
      BXEQ lr

      ADDS r3, r3, #1 ; Loop index ++
      CMP r3, r1 ; r1 = array size
      BCC loop

      MOV r0, #0xFFFFFFFF
      BX LR ; return -1 if failed

ENDP
```